Cape Peninsula University of Technology

Faculty of Engineering and the Built Environment
Department of Electrical, Electronic and Computer Engineering

Software Design 1 (SDN150S)
Project Brief
Due Date – November 08, 2026

Group member	**Student number**	**File**
Muvhofhololi Tsanwani	261021222	Account, MainMenuFrame, WithdrawFrame;
Aubrey Magolego	231255640	ATMStyle
Kgonyane Makgoale	260763314	Bank
Mabandla Nkosi	260407763	LoginFrame
Vhukhudo Ravele	260048496	DepositFrame, README
Athenkosi Khau	215107020	BalanceFrame, report
Siyolise Mhlauli	261295438	Main, ATMLogicTest
Ndzalama Mthombeni	260749273	Transaction, InvalidAmountException, InsufficientFundsException

Source code: https://github.com/muvhofhololitsanwani-design/atm-project-group2

1. Introduction

An automated teller machine (ATM) lets customers withdraw cash, deposit money and check their balance without visiting a branch, which makes formal financial services more accessible, especially in rural areas. This report explains how Group 2 designed and implemented a desktop ATM simulator in Java. After authenticating, the user chooses Withdraw, Deposit, Check Balance or Exit, and the account starts with R10 000.00. The program had to use constructors, modifiers, control statements and JFrame forms, with error handling and input validation. The main idea is that keeping the rules separate from the screens, in an object-oriented design, produced a reliable and testable ATM that eight members could build in parallel. The report covers our sub-questions, design approach, engineering method, skills and tools, the pseudocode and UML class diagram, the test results and our recommendations.

2. Sub-questions

We broke the problem into six sub-questions:

1. **Structure:** which classes are needed and how do they cooperate? (Sections 3.1 and 3.7)
2. **Security:** how is a customer authenticated and the account protected? (Sections 3.5 and 3.6)
3. **Correctness:** how are withdrawals and deposits validated so the balance is never corrupted? (Sections 3.1 and 3.5)
4. **Errors:** how are mistakes reported without crashing the program? (Section 3.5)
5. **Interface:** how is a user-friendly GUI built with JFrame forms? (Sections 3.5 and 4.1)
6. **Teamwork:** how do eight members share the work and keep it traceable? (Section 3.2)

3. Discussion

3.1 Software design approach

We used object-oriented analysis and design, because the problem contains natural objects: accounts, a bank, transactions and screens. Responsibilities were separated into three groups. The **model** classes (Account, Bank, Transaction) hold the data and enforce the rules, and know nothing about Swing. The **user-interface** classes (LoginFrame, MainMenuFrame, WithdrawFrame, DepositFrame, BalanceFrame) show information and collect input. The **support** classes are ATMStyle (colours, fonts and helper methods) and two custom exceptions. Because the rules live outside the screens, we could test them without opening a window.

All fields are private and reached through methods (encapsulation). Each screen extends JFrame and implements ActionListener, so it reacts to button clicks. The course notes warn that double values

should not be used for currency, so Account stores the balance as whole cents in a long variable and converts only for display. This answers the correctness sub-question.

3.2 Engineering method

We followed a simplified software development life cycle: requirements analysis from the project brief, design (pseudocode and a UML class diagram), implementation in small increments, testing after each increment, and integration through Git. The model classes were built first, then the login screen, the main menu and the three transaction screens. A shared Git repository on GitHub kept the work traceable: each member owns specific files, works on a branch, commits and opens a pull request for the group leader to review. The title page lists each member's files.

3.3 Skills applied

The project drew on Java programming, object-oriented concepts (classes, inheritance, polymorphism, encapsulation), exception handling, event-driven GUI design, algorithm design in pseudocode, UML modelling, testing and debugging, version control, teamwork and technical writing.

3.4 Tools and their limitations

Table 1: Tools used, their purpose and limitations

Tool	Purpose	Limitation
JDK (Java SE)	Compiles and runs the program	Must be installed on every computer
VS Code with Java extensions	Editing, running and debugging	No visual form designer; every form is coded by hand
Java Swing (Oracle, n.d.)	GUI toolkit: JFrame, JButton, JTextField, JTable	Dated appearance; all screen updates must happen on one thread
Git and GitHub	Version control, branches, pull requests	Needs discipline: small commits and clear messages
ATMLogicTest	Console test program with 24 checks	Written by hand; a framework such as JUnit would scale better
Graphviz	Draws the UML diagram from a text file	Layout is automatic, so some arrows cross

3.5 Implementation of the required concepts

- **Constructors:** every class has one. Account has two; the shorter one calls the longer with this(...) and supplies the opening balance of R10 000.00.
- **Modifiers:** private fields; public constructors and methods; static final constants (INITIAL_BALANCE, MAX_WITHDRAWAL, MAX_DEPOSIT, MAX_PIN_ATTEMPTS); static helper methods and a private constructor in ATMStyle.
- **Control statements:** if / else if chains validate input; a switch handles the four login results; for loops search the account list and build the mini statement; try / catch blocks handle exceptions.

- **JFrame forms:** five JFrame screens. The login screen closes itself and opens the main menu; the menu hides while a transaction screen is open, and each Back button closes that screen and shows the menu again.
- **Error handling and validation:** the screens first check the format (10-digit account number, 4-digit PIN, numeric amount). Account then enforces the rules and throws InvalidAmountException or InsufficientFundsException, which are custom exceptions that extend Exception.

Business rules: withdrawals are multiples of R10 up to R5 000; deposits are up to R20 000; amounts have at most two decimals; three wrong PINs block the card. Code excerpts are in Appendix A.

3.6 Pseudocode

```
START
  Create the bank with three demo accounts; show the login screen
  LOGIN: read account number and PIN
    IF a field is empty or has the wrong number of digits THEN show a message
    ELSE find the account
      IF it does not exist THEN show "Account not found"
      ELSE IF the card is blocked THEN show "Card blocked"
      ELSE IF the PIN is wrong THEN
        add 1 to the failed attempts; block the card at 3; show attempts left
      ELSE reset the failed attempts and open the main menu
  MENU: show Withdraw, Deposit, Check Balance, Exit
    WITHDRAW: read the amount
      IF it is not a valid number THEN error
      ELSE IF it is not a multiple of R10 THEN error
      ELSE IF it is more than R5 000 THEN error
      ELSE IF it is more than the balance THEN "Insufficient funds"
      ELSE subtract it, record the transaction, show success message
    DEPOSIT: read the amount
      IF it is not valid or more than R20 000 THEN error
      ELSE add it, record the transaction, show success message
    CHECK BALANCE: show the balance and the last 5 transactions
    EXIT: ask for confirmation, then close the program
END
```

3.7 UML class diagram

Figures 1 and 2 show the class diagram in two parts: the model classes (Figure 1) and the screens (Figure 2). Inheritance is drawn with a hollow triangle. Bank is an aggregate of Account objects and

Account is composed of its Transaction history. Account throws the two custom exceptions, and the frames depend on Account and on each other as the user moves from screen to screen. Each transaction screen keeps a reference to the main menu so that its Back button can return to it. ATMStyle is used by the screens and for formatting money in the model classes; its arrows are omitted to keep the diagram readable.

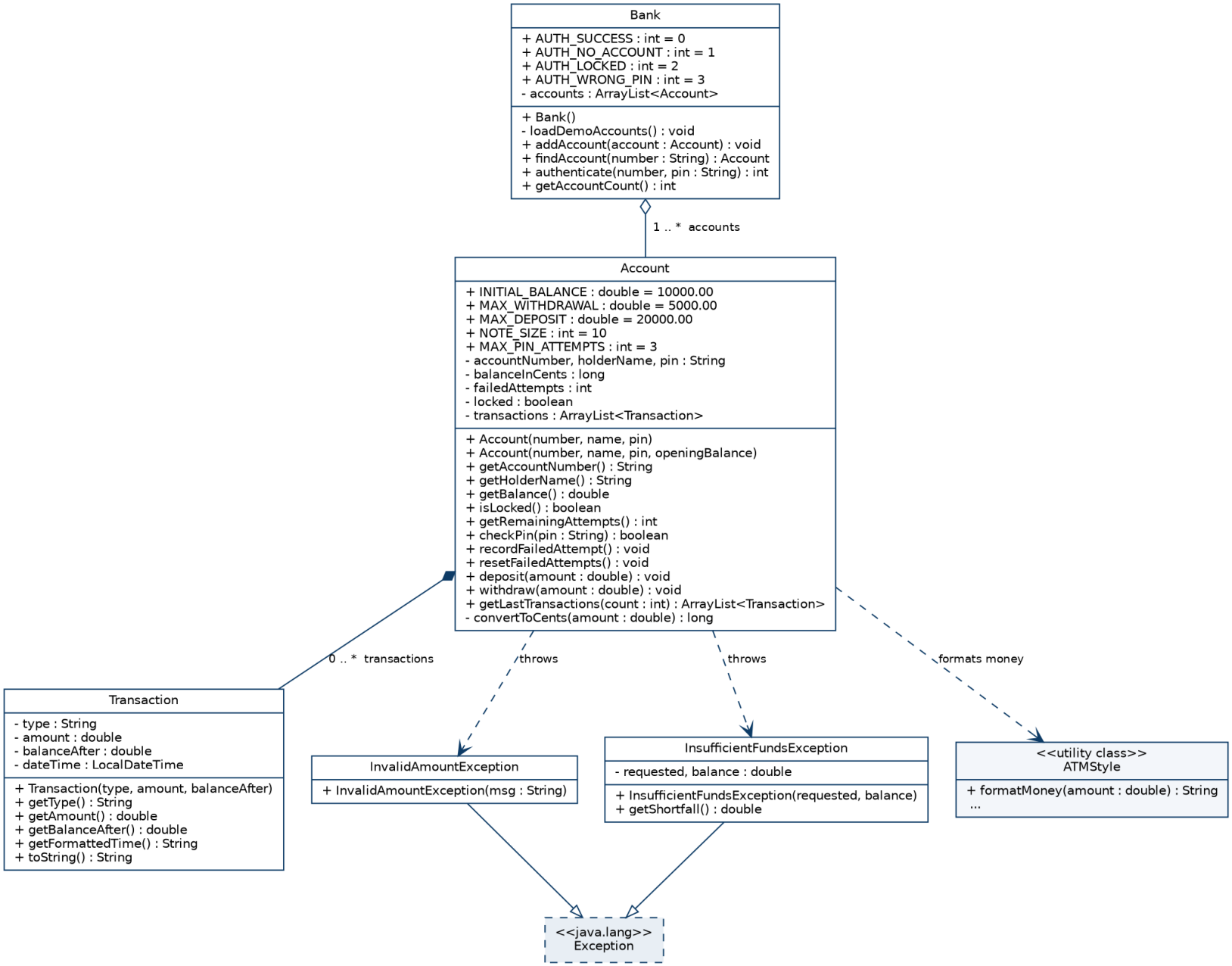


Figure 1: UML class diagram, part 1 – the model classes

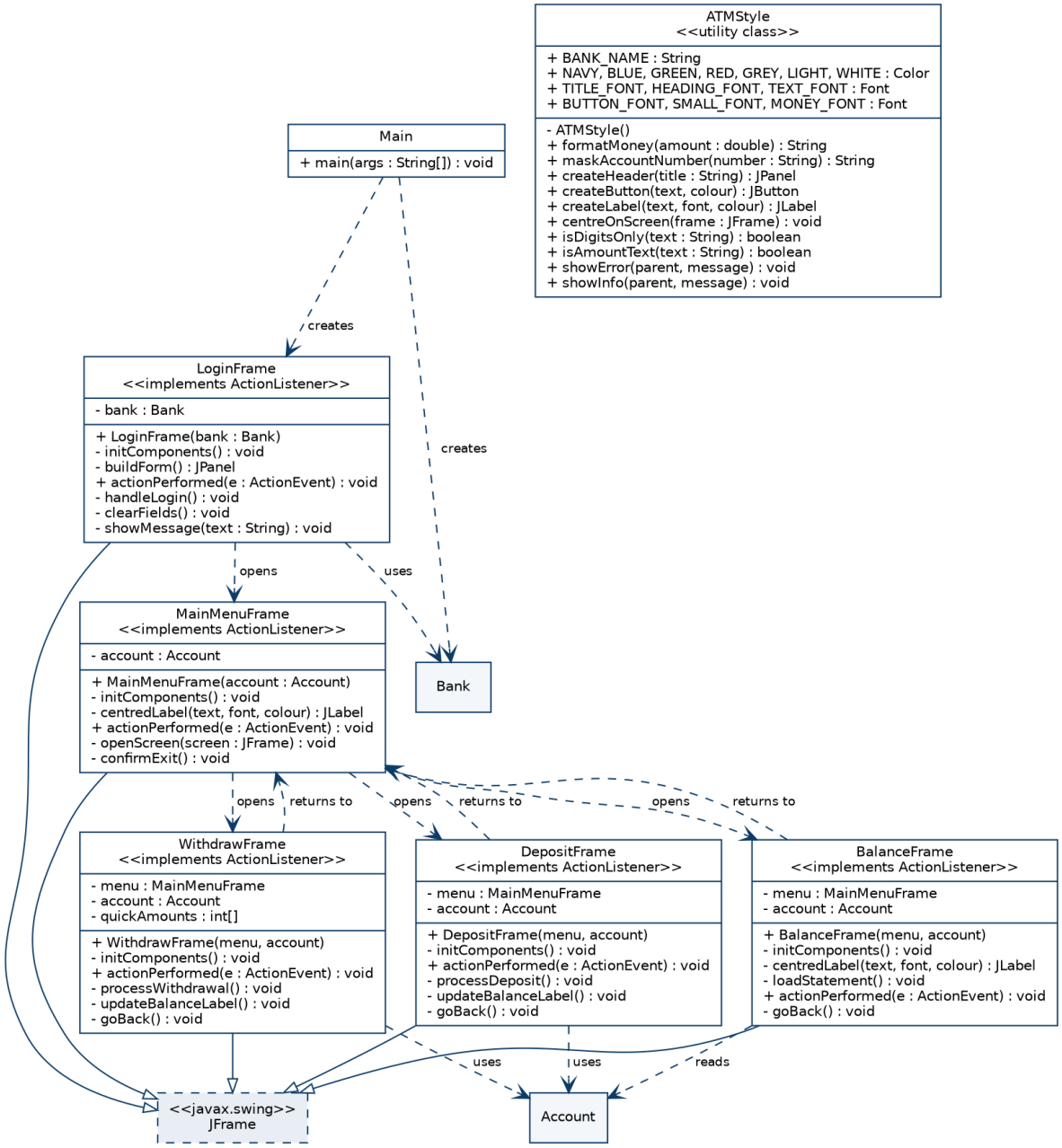


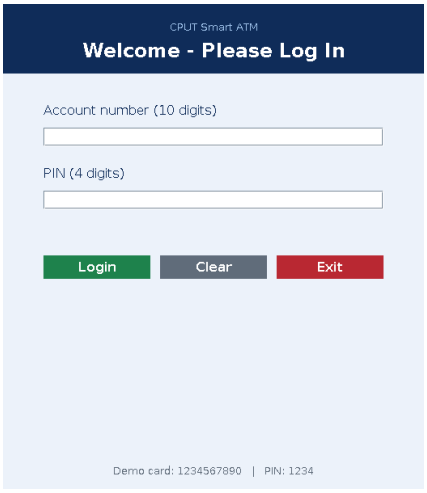
Figure 2: UML class diagram, part 2 – Main and the five screens; GUI component fields (buttons, text fields, labels) are omitted

4. Results and analysis

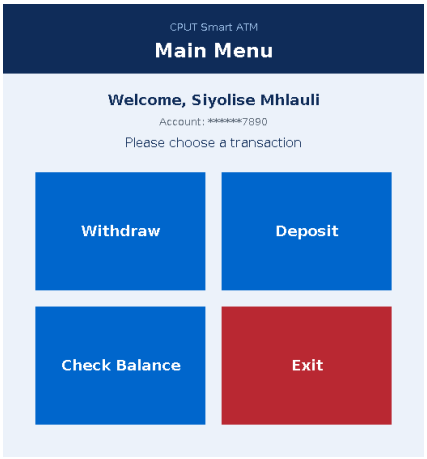
4.1 Testing

Table 2: Test cases for the screens

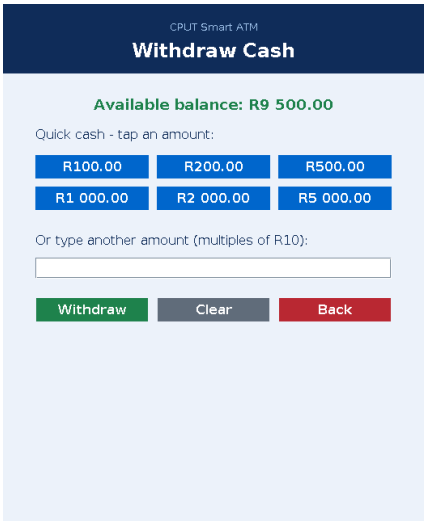
ID	Test	Input	Expected result	Result
T1	Valid login	1234567890 / 1234	Main menu opens	Pass
T2	Wrong PIN	1234567890 / 0000	Message shows 2 attempts left	Pass
T3	Card blocking	3 wrong PINs, then the correct PIN	Card blocked; correct PIN refused	Pass
T4	Valid withdrawal	R500	Success message; balance R9 500.00	Pass
T5	Invalid withdrawals	empty, abc, 15, 0, -50, 12.345, 6000	Error message; balance unchanged	Pass
T6	Insufficient funds	R500 from an account with R100 (ATMLogicTest)	InsufficientFundsException; shortfall R400.00	Pass
T7	Valid deposit	R250.50	Success message; balance R9 750.50	Pass
T8	Invalid deposits	empty, xyz, 0, -10, 25000, 10.999	Error message; balance unchanged	Pass
T9	Mini statement	After T4 and T7	Withdrawal R500.00 and Deposit R250.50 listed	Pass
T10	Navigation	Back button on each screen	Returns to the main menu	Pass



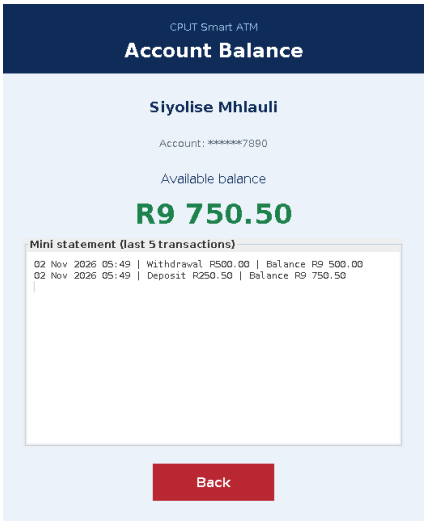
(a) Login screen



(b) Main menu



(c) Withdraw screen after R500 withdrawn



(d) Balance and mini statement

Figure 3: Screenshots of the running ATM simulator

4.2 Analysis

All requirements of the brief were met: authentication, the R10 000.00 opening balance, withdraw, deposit, balance and exit with System.exit(0), together with validation and error handling. Three design decisions proved useful. Keeping the rules in Account allowed fast console testing. Storing cents avoids rounding errors, so R250.50 is added exactly. Validating twice, once on the screen and again in Account, means a mistake in one layer cannot corrupt the balance.

The tools and assumptions also have limits. Balances are held in memory, so they reset when the program closes. PINs and the three demo accounts are stored as plain text in the source code, which would be unacceptable in a real bank, where PINs must be hashed and accounts kept in a secure database. The program serves one user at a time, there is no cash-note inventory, and Swing windows look dated. Our tests are automated for the rules but manual for the screens.

5. Conclusion

Group 2 delivered a working Java Swing ATM that meets the brief and demonstrates constructors, modifiers, control statements and JFrame forms. The object-oriented design separated rules from screens, made testing easy and allowed eight members to work on separate files through Git. We recommend three improvements: store accounts and transactions in a file or database, hash the PINs, and replace the console tests with JUnit tests.

References

Cape Peninsula University of Technology. 2026 Software Design 1 project brief.

Cape Peninsula University of Technology. n.d. Software Design 1: lecture notes on datatypes, exception handling and Swing GUI.

Chacon, S. & Straub, B. 2014. *Pro Git*. 2nd ed. New York: Apress.

Deitel, P. & Deitel, H. 2017. *Java how to program*. 11th ed. Hoboken, NJ: Pearson.

Fowler, M. 2004. *UML distilled: a brief guide to the standard object modeling language*. 3rd ed. Boston, MA: Addison-Wesley.

Oracle. n.d. Creating a GUI with Swing. The Java tutorials. https://docs.oracle.com/javase/tutorial/uiswing

Appendix A: Code excerpts

A.1 Account.withdraw() – rules and custom exceptions

```
public void withdraw(double amount)
      throws InvalidAmountException, InsufficientFundsException {
   long cents = convertToCents(amount);       // validates the amount first
   // Rule 1: the ATM only dispenses whole notes (multiples of R10)
   if (cents % (NOTE_SIZE * 100L) != 0) {
      throw new InvalidAmountException(/* "enter a multiple of R10" */);
   }
   // Rule 2: maximum withdrawal per transaction
   if (cents > MAX_WITHDRAWAL * 100) {
      throw new InvalidAmountException(/* "maximum withdrawal is R5 000" */);
   }
   // Rule 3: the customer cannot withdraw more than the balance
   if (cents > balanceInCents) {
      throw new InsufficientFundsException(cents / 100.0, getBalance());
   }
   balanceInCents = balanceInCents - cents;    // deduct from the balance
   transactions.add(new Transaction("Withdrawal", cents / 100.0, getBalance()));
}
```

A.2 LoginFrame.handleLogin() – validation and switch (extract)

```
if (!ATMStyle.isDigitsOnly(accountNumber) || accountNumber.length() != 10) {
   showMessage("The account number must be exactly 10 digits.");
   return;
}
int result = bank.authenticate(accountNumber, pin);
switch (result) {
   case Bank.AUTH_SUCCESS: {
      Account account = bank.findAccount(accountNumber);
      MainMenuFrame menu = new MainMenuFrame(account);   // next screen
      menu.setVisible(true);
```

```
        dispose();                          // close login
        break;
    }
    case Bank.AUTH_NO_ACCOUNT:
        showMessage("Account not found. Please check the number.");
        break;
    case Bank.AUTH_WRONG_PIN:
        showMessage("Incorrect PIN. Attempts left: " + account.getRemainingAttempts());
        break;
    case Bank.AUTH_LOCKED:
        showMessage("This card is blocked.");
        break;
}
```

A.3 WithdrawFrame.processWithdrawal() – try / catch (extract)

```
try {

    double amount = Double.parseDouble(text);

    account.withdraw(amount);               // may throw 2 exceptions

    updateBalanceLabel();

    ATMStyle.showInfo(this, "Withdrawal successful! ...");

} catch (NumberFormatException ex) {

    ATMStyle.showError(this, "That is not a valid number.");

} catch (InvalidAmountException ex) {

    ATMStyle.showError(this, ex.getMessage());

} catch (InsufficientFundsException ex) {

    ATMStyle.showError(this, ex.getMessage());

}
```

Appendix B: Complete source code

B.1 Main.java

```
/*
 *  Author      : Siyolise Mhlauli
 *  Student No. : 261295438
 */
package atm;

import javax.swing.SwingUtilities;
import javax.swing.UIManager;
```

```
public class Main {

   public static void main(String[] args) {

      // Force cross-platform look and feel so UI colors and borders across operating systems
      try {
         UIManager.setLookAndFeel(UIManager.getCrossPlatformLookAndFeelClassName());
      } catch (Exception e) {
         System.out.println("Could not set look and feel: " + e.getMessage());
      }

      // Launch the application GUI on the Swing Event Dispatch Thread (EDT)
      SwingUtilities.invokeLater(new Runnable() {
         @Override
         public void run() {
            Bank bank = new Bank();            // Create bank instance and load demo accounts
            LoginFrame login = new LoginFrame(bank); // Instantiate and display the initial login screen
            login.setVisible(true);
         }
      });
   }
}
```

B.2 ATMStyle.java

```
/*  Student Number: 231255640
 *  Author: Aubrey Magolego
 */
package atm;

import java.awt.Color;
import java.awt.Component;
import java.awt.Cursor;
import java.awt.Dimension;
import java.awt.Font;
import java.awt.GridLayout;
import java.awt.Toolkit;
import java.awt.event.MouseAdapter;
import java.awt.event.MouseEvent;
import java.util.Locale;
import javax.swing.BorderFactory;
import javax.swing.JButton;
import javax.swing.JFrame;
import javax.swing.JLabel;
import javax.swing.JOptionPane;
import javax.swing.JPanel;
import javax.swing.SwingConstants;

public final class ATMStyle {

   public static final String BANK_NAME = "CPUT Smart ATM";

   // Application color scheme constants
```

```
public static final Color NAVY  = new Color(15, 44, 89);
public static final Color BLUE  = new Color(0, 102, 204);
public static final Color GREEN = new Color(30, 130, 76);
public static final Color RED   = new Color(185, 40, 50);
public static final Color GREY  = new Color(96, 108, 122);
public static final Color LIGHT = new Color(236, 242, 250);
public static final Color WHITE = Color.WHITE;

// Standardized fonts across all ATM frames
public static final Font TITLE_FONT   = new Font("SansSerif", Font.BOLD, 24);
public static final Font HEADING_FONT = new Font("SansSerif", Font.BOLD, 18);
public static final Font TEXT_FONT    = new Font("SansSerif", Font.PLAIN, 16);
public static final Font BUTTON_FONT = new Font("SansSerif", Font.BOLD, 16);
public static final Font SMALL_FONT   = new Font("SansSerif", Font.PLAIN, 13);
public static final Font MONEY_FONT   = new Font("SansSerif", Font.BOLD, 34);

// Private constructor to prevent instantiation of this utility class
private ATMStyle() {
}

// Formats numerical amounts into South African currency format (e.g., R10 000.00)
public static String formatMoney(double amount) {
   String text = String.format(Locale.US, "%,.2f", amount);
   return "R" + text.replace(",", " ");
}

// Masks sensitive account numbers for display, leaving only the last 4 digits visible
public static String maskAccountNumber(String accountNumber) {
   if (accountNumber.length() <= 4) {
      return accountNumber;
   }
   String stars = "";
   for (int i = 0; i < accountNumber.length() - 4; i++) {
      stars = stars + "*";
   }
   return stars + accountNumber.substring(accountNumber.length() - 4);
}

// Creates the standard navy header panel used at the top of GUI screens
public static JPanel createHeader(String screenTitle) {
   JPanel header = new JPanel(new GridLayout(2, 1));
   header.setBackground(NAVY);
   header.setBorder(BorderFactory.createEmptyBorder(14, 10, 14, 10));

   JLabel bankLabel = new JLabel(BANK_NAME, SwingConstants.CENTER);
   bankLabel.setFont(SMALL_FONT);
   bankLabel.setForeground(new Color(160, 195, 235));

   JLabel titleLabel = new JLabel(screenTitle, SwingConstants.CENTER);
   titleLabel.setFont(TITLE_FONT);
   titleLabel.setForeground(WHITE);

   header.add(bankLabel);
   header.add(titleLabel);
```

```
    return header;
}

// Helper to build styled buttons with custom hover color effects
public static JButton createButton(String text, final Color background) {
    final JButton button = new JButton(text);
    button.setFont(BUTTON_FONT);
    button.setForeground(WHITE);
    button.setBackground(background);
    button.setOpaque(true);
    button.setBorderPainted(false);
    button.setFocusPainted(false);
    button.setCursor(Cursor.getPredefinedCursor(Cursor.HAND_CURSOR));
    button.setPreferredSize(new Dimension(150, 46));

    button.addMouseListener(new MouseAdapter() {
        @Override
        public void mouseEntered(MouseEvent e) {
            button.setBackground(background.darker());
        }

        @Override
        public void mouseExited(MouseEvent e) {
            button.setBackground(background);
        }
    });
    return button;
}

public static JLabel createLabel(String text, Font font, Color colour) {
    JLabel label = new JLabel(text);
    label.setFont(font);
    label.setForeground(colour);
    return label;
}

// Centers any JFrame window on the user's desktop screen
public static void centreOnScreen(JFrame frame) {
    Dimension screen = Toolkit.getDefaultToolkit().getScreenSize();
    int x = (screen.width - frame.getWidth()) / 2;
    int y = (screen.height - frame.getHeight()) / 2;
    frame.setLocation(x, y);
}

// Validates that user input fields contain strictly numeric characters
public static boolean isDigitsOnly(String text) {
    if (text.length() == 0) {
        return false;
    }
    for (int i = 0; i < text.length(); i++) {
        char c = text.charAt(i);
        if (c < '0' || c > '9') {
            return false;
        }
```

```
        }
        return true;
    }

    // Validates deposit and withdrawal text inputs to ensure correct decimal formatting
    public static boolean isAmountText(String text) {
        if (text.length() == 0 || text.equals(".")) {
            return false;
        }
        int dotCount = 0;
        for (int i = 0; i < text.length(); i++) {
            char c = text.charAt(i);
            if (c == '.') {
                dotCount++;
            } else if (c < '0' || c > '9') {
                return false;
            }
        }
        return dotCount <= 1;
    }

    public static void showError(Component parent, String message) {
        JOptionPane.showMessageDialog(parent, message, "Error", JOptionPane.ERROR_MESSAGE);
    }

    public static void showInfo(Component parent, String message) {
        JOptionPane.showMessageDialog(parent, message, BANK_NAME,
JOptionPane.INFORMATION_MESSAGE);
    }
}
```

B.3 Account.java

```
/*
 *  Author      : Muvhofhololi Tsanwani
 *  Student No. : 261021222
 */
package atm;

import java.util.ArrayList;

public class Account
{
    public static final double INITIAL_BALANCE = 10000.00;
    public static final double MAX_WITHDRAWAL  = 5000.00;
    public static final double MAX_DEPOSIT     = 20000.00;
    public static final int    NOTE_SIZE       = 10;
    public static final int    MAX_PIN_ATTEMPTS = 3;

    private final String accountNumber;
    private final String holderName;
    private final String pin;
    private long balanceInCents; // Stored in cents to prevent floating-point rounding errors
```

```
private final ArrayList<Transaction> transactions;
private int failedAttempts;
private boolean locked;

// Standard account creation with the default R10 000.00 opening balance
public Account(String accountNumber, String holderName, String pin) {
   this(accountNumber, holderName, pin, INITIAL_BALANCE);
}

// Account creation with a custom opening balance
public Account(String accountNumber, String holderName, String pin, double openingBalance) {
   this.accountNumber = accountNumber;
   this.holderName = holderName;
   this.pin = pin;
   this.balanceInCents = Math.round(openingBalance * 100);
   this.transactions = new ArrayList<>();
   this.failedAttempts = 0;
   this.locked = false;
}

public String getAccountNumber() {
   return accountNumber;
}

public String getHolderName() {
   return holderName;
}

// Converts balance back to Rands for display
public double getBalance() {
   return (double) balanceInCents / 100.0;
}

public boolean isLocked() {
   return locked;
}

public int getRemainingAttempts() {
   return MAX_PIN_ATTEMPTS - failedAttempts;
}

public boolean checkPin(String enteredPin) {
   return pin.equals(enteredPin);
}

public void recordFailedAttempt() {
   failedAttempts++;
   if (failedAttempts >= MAX_PIN_ATTEMPTS) {
      locked = true;
   }
}

public void resetFailedAttempts() {
   failedAttempts = 0;
```

```
    }

    public void deposit(double amount) throws InvalidAmountException {
        long cents = convertToCents(amount);

        if (cents > (long) (MAX_DEPOSIT * 100)) {
            throw new InvalidAmountException("The maximum deposit per transaction is "
                    + ATMStyle.formatMoney(MAX_DEPOSIT) + ".");
        }

        balanceInCents += cents;
        transactions.add(new Transaction("Deposit", (double) cents / 100.0, getBalance()));
    }

    public void withdraw(double amount) throws InvalidAmountException, InsufficientFundsException
{
        long cents = convertToCents(amount);

        // Ensure the amount matches the ATM note denominations (multiples of R10)
        if (cents % (NOTE_SIZE * 100L) != 0) {
            throw new InvalidAmountException("Please enter a multiple of R" + NOTE_SIZE
                    + ". The ATM only has R10, R20, R50, R100 and R200 notes.");
        }

        if (cents > (long) (MAX_WITHDRAWAL * 100)) {
            throw new InvalidAmountException("The maximum withdrawal per transaction is "
                    + ATMStyle.formatMoney(MAX_WITHDRAWAL) + ".");
        }

        if (cents > balanceInCents) {
            throw new InsufficientFundsException((double) cents / 100.0, getBalance());
        }

        balanceInCents -= cents;
        transactions.add(new Transaction("Withdrawal", (double) cents / 100.0, getBalance()));
    }

    // Returns recent transactions for the mini statement display
    public ArrayList<Transaction> getLastTransactions(int count) {
        ArrayList<Transaction> recent = new ArrayList<>();
        int start = transactions.size() - count;
        if (start < 0) {
            start = 0;
        }
        for (int i = start; i < transactions.size(); i++) {
            recent.add(transactions.get(i));
        }
        return recent;
    }

    // Validates the amount and converts it to cents to avoid precision errors
    private long convertToCents(double amount) throws InvalidAmountException {
        if (amount <= 0) {
            throw new InvalidAmountException("The amount must be greater than R0.00.");
```

```
        }
        return Math.round(amount * 100);
    }
}
```

B.4 Bank.java

```
/* Student Number;260763314
 * Written by; Kgonyane Makgoale
 */
package atm;

import java.util.ArrayList;

public class Bank {

    // Authentication result codes
    public static final int AUTH_SUCCESS    = 0;
    public static final int AUTH_NO_ACCOUNT = 1;
    public static final int AUTH_LOCKED     = 2;
    public static final int AUTH_WRONG_PIN  = 3;

    private final ArrayList<Account> accounts;

    public Bank() {
        accounts = new ArrayList<>();
        loadDemoAccounts();
    }

    // Loads predefined demo accounts for testing and demonstration purposes
    private void loadDemoAccounts() {
        addAccount(new Account("1234567890", "Siyolise Mhlauli", "1234"));
        addAccount(new Account("0987654321", "Athenkosi Khau", "4321"));
        addAccount(new Account("1122334455", "Kgonyane Makgoale", "2468"));
    }

    public void addAccount(Account account) {
        accounts.add(account);
    }

    // Searches for an account by its number, returning null if it doesn't exist
    public Account findAccount(String accountNumber) {
        for (Account account : accounts) {
            if (account.getAccountNumber().equals(accountNumber)) {
                return account;
            }
        }
        return null;
    }

    // Validates login credentials and enforces the 3-attempt card lockout rule
    public int authenticate(String accountNumber, String pin) {
        Account account = findAccount(accountNumber);
```

```
        if (account == null) {
            return AUTH_NO_ACCOUNT;
        }
        if (account.isLocked()) {
            return AUTH_LOCKED;
        }
        if (account.checkPin(pin)) {
            account.resetFailedAttempts(); // Clear failed count upon successful login
            return AUTH_SUCCESS;
        }

        // Increment failed attempt counter; blocks the card on the 3rd failure
        account.recordFailedAttempt();
        if (account.isLocked()) {
            return AUTH_LOCKED;
        }
        return AUTH_WRONG_PIN;
    }

    public int getAccountCount() {
        return accounts.size();
    }
}
```

B.5 Transaction.java

```
/*
 *  Author      : Ndzalama Mthombeni
 *  Student No. : 260749273
 */
package atm;

import java.time.LocalDateTime;
import java.time.format.DateTimeFormatter;
import java.util.Locale;

public class Transaction {

    // Date and time formatter for transaction logging and mini statements (e.g., 28 Oct 2026 14:05)
    private static final DateTimeFormatter TIME_FORMAT =
            DateTimeFormatter.ofPattern("dd MMM yyyy HH:mm", Locale.ENGLISH);

    private final String type;              // "Deposit" or "Withdrawal"
    private final double amount;            // Transaction amount in Rands
    private final double balanceAfter;      // Resulting account balance after transaction
    private final LocalDateTime dateTime;   // Timestamp captured automatically upon creation

    // Constructor automatically records the exact timestamp of the transaction
    public Transaction(String type, double amount, double balanceAfter) {
        this.type = type;
        this.amount = amount;
        this.balanceAfter = balanceAfter;
```

```
        this.dateTime = LocalDateTime.now();
    }

    public String getType() {
        return type;
    }

    public double getAmount() {
        return amount;
    }

    public double getBalanceAfter() {
        return balanceAfter;
    }

    public String getFormattedTime() {
        return dateTime.format(TIME_FORMAT);
    }

    // Formats transaction details into a single structured line for the mini statement display
    @Override
    public String toString() {
        return getFormattedTime() + " | " + type + " " + ATMStyle.formatMoney(amount)
                + " | Balance " + ATMStyle.formatMoney(balanceAfter);
    }
}
```

B.6 InvalidAmountException.java

```
/*
 *  Author      : Ndzalama Mthombeni
 *  Student No. : 260749273
 */
package atm;

public class InvalidAmountException extends Exception {

    // Passes the error description to the Exception superclass
    public InvalidAmountException(String message) {
        super(message);
    }
}
```

B.7 InsufficientFundsException.java

```
/*
 *  Author      : Ndzalama Mthombeni
 *  Student No. : 260749273
 */
package atm;

public class InsufficientFundsException extends Exception {
```

```
    private final double requested;   // the amount the customer asked for
    private final double balance;     // the balance that was available

    public InsufficientFundsException(double requested, double balance) {
        super("Insufficient funds. You asked for " + ATMStyle.formatMoney(requested)
                + " but your available balance is " + ATMStyle.formatMoney(balance) + ".");
        this.requested = requested;
        this.balance = balance;
    }

    public double getRequested() {
        return requested;
    }

    public double getBalance() {
        return balance;
    }

    // Calculates how much additional money is needed to cover the requested withdrawal
    public double getShortfall() {
        return requested - balance;
    }
}
```

B.8 LoginFrame.java

```
// Author: Mabandla Nkosi
// Student number: 260407763

package atm;

import java.awt.BorderLayout;
import java.awt.Dimension;
import java.awt.GridBagConstraints;
import java.awt.GridBagLayout;
import java.awt.GridLayout;
import java.awt.Insets;
import java.awt.event.ActionEvent;
import java.awt.event.ActionListener;
import javax.swing.BorderFactory;
import javax.swing.JButton;
import javax.swing.JFrame;
import javax.swing.JLabel;
import javax.swing.JPanel;
import javax.swing.JPasswordField;
import javax.swing.JTextField;
import javax.swing.SwingConstants;

public class LoginFrame extends JFrame implements ActionListener {

    private final Bank bank;
    private JTextField accountField;
    private JPasswordField pinField;
```

```
    private JButton loginButton;
    private JButton clearButton;
    private JButton exitButton;
    private JLabel messageLabel;

    public LoginFrame(Bank bank) {
        this.bank = bank;
        initComponents();
    }

    private void initComponents() {
        setTitle("CPUT Smart ATM - Login");
        setSize(520, 600);
        setResizable(false);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setLayout(new BorderLayout());
        getContentPane().setBackground(ATMStyle.LIGHT);

        add(ATMStyle.createHeader("Welcome - Please Log In"), BorderLayout.NORTH);
        add(buildForm(), BorderLayout.CENTER);

        // Quick demo hint displayed at the bottom for testing and grading purposes
        JLabel hintLabel = new JLabel("Demo card: 1234567890   |   PIN: 1234",
SwingConstants.CENTER);
        hintLabel.setFont(ATMStyle.SMALL_FONT);
        hintLabel.setForeground(ATMStyle.GREY);
        hintLabel.setBorder(BorderFactory.createEmptyBorder(8, 0, 14, 0));
        add(hintLabel, BorderLayout.SOUTH);

        getRootPane().setDefaultButton(loginButton);   // Pressing Enter triggers the login button
        ATMStyle.centreOnScreen(this);
    }

    private JPanel buildForm() {
        JPanel form = new JPanel(new GridBagLayout());
        form.setBackground(ATMStyle.LIGHT);
        form.setBorder(BorderFactory.createEmptyBorder(30, 50, 10, 50));

        GridBagConstraints gbc = new GridBagConstraints();
        gbc.gridx = 0;
        gbc.weightx = 1.0;
        gbc.fill = GridBagConstraints.HORIZONTAL;
        gbc.anchor = GridBagConstraints.WEST;
        gbc.insets = new Insets(6, 0, 6, 0);

        // Account number field label and text input
        gbc.gridy = 0;
        form.add(ATMStyle.createLabel("Account number (10 digits)", ATMStyle.TEXT_FONT,
ATMStyle.NAVY), gbc);

        accountField = new JTextField();
        accountField.setFont(ATMStyle.TEXT_FONT);
        accountField.setPreferredSize(new Dimension(300, 42));
        gbc.gridy = 1;
```

```
    form.add(accountField, gbc);

    // PIN field label and password input
    gbc.gridy = 2;
    gbc.insets = new Insets(18, 0, 6, 0);
    form.add(ATMStyle.createLabel("PIN (4 digits)", ATMStyle.TEXT_FONT, ATMStyle.NAVY), gbc);

    pinField = new JPasswordField();
    pinField.setFont(ATMStyle.TEXT_FONT);
    pinField.setPreferredSize(new Dimension(300, 42));
    gbc.gridy = 3;
    gbc.insets = new Insets(6, 0, 6, 0);
    form.add(pinField, gbc);

    // Error or status message label
    messageLabel = ATMStyle.createLabel(" ", ATMStyle.SMALL_FONT, ATMStyle.RED);
    gbc.gridy = 4;
    gbc.insets = new Insets(12, 0, 12, 0);
    form.add(messageLabel, gbc);

    // Action buttons layout
    loginButton = ATMStyle.createButton("Login", ATMStyle.GREEN);
    clearButton = ATMStyle.createButton("Clear", ATMStyle.GREY);
    exitButton  = ATMStyle.createButton("Exit", ATMStyle.RED);
    loginButton.addActionListener(this);
    clearButton.addActionListener(this);
    exitButton.addActionListener(this);

    JPanel buttonPanel = new JPanel(new GridLayout(1, 3, 12, 0));
    buttonPanel.setBackground(ATMStyle.LIGHT);
    buttonPanel.add(loginButton);
    buttonPanel.add(clearButton);
    buttonPanel.add(exitButton);
    gbc.gridy = 5;
    gbc.insets = new Insets(10, 0, 0, 0);
    form.add(buttonPanel, gbc);

    // Vertical glue constraint to push form components toward the top
    gbc.gridy = 6;
    gbc.weighty = 1.0;
    form.add(new JLabel(), gbc);

    return form;
}

@Override
public void actionPerformed(ActionEvent e) {
    Object source = e.getSource();

    if (source == loginButton) {
        handleLogin();
    } else if (source == clearButton) {
        clearFields();
    } else if (source == exitButton) {
```

```
        System.exit(0);
    }
}

// Validates inputs and handles authentication through the Bank model
private void handleLogin() {
    String accountNumber = accountField.getText().trim();
    String pin = new String(pinField.getPassword());

    // Basic format and presence validation
    if (accountNumber.isEmpty() || pin.isEmpty()) {
        showMessage("Please enter your account number and PIN.");
        return;
    }
    if (!ATMStyle.isDigitsOnly(accountNumber) || accountNumber.length() != 10) {
        showMessage("The account number must be exactly 10 digits.");
        return;
    }
    if (!ATMStyle.isDigitsOnly(pin) || pin.length() != 4) {
        showMessage("The PIN must be exactly 4 digits.");
        return;
    }

    // Authenticate credentials against the bank registry
    int result = bank.authenticate(accountNumber, pin);

    switch (result) {
        case Bank.AUTH_SUCCESS: {
            Account account = bank.findAccount(accountNumber);
            MainMenuFrame menu = new MainMenuFrame(account);   // Open main menu screen
            menu.setVisible(true);
            dispose();                              // Close login window
            break;
        }
        case Bank.AUTH_NO_ACCOUNT:
            showMessage("Account not found. Please check the number.");
            break;
        case Bank.AUTH_WRONG_PIN: {
            Account account = bank.findAccount(accountNumber);
            showMessage("Incorrect PIN. Attempts left: " + account.getRemainingAttempts());
            pinField.setText("");
            break;
        }
        case Bank.AUTH_LOCKED:
            pinField.setText("");
            showMessage("This card is blocked.");
            ATMStyle.showError(this, "Your card has been blocked after "
                    + Account.MAX_PIN_ATTEMPTS + " wrong PIN attempts.\n"
                    + "Please contact your bank.\n"
                    + "(Demo: restart the program to unblock the card.)");
            break;
        default:
            showMessage("Unknown error. Please try again.");
            break;
```

```
        }
    }

    private void clearFields() {
        accountField.setText("");
        pinField.setText("");
        messageLabel.setText(" ");
        accountField.requestFocus();
    }

    private void showMessage(String text) {
        messageLabel.setText(text);
    }
}
```

B.9 MainMenuFrame.java

```
/*
 *  Author: Muvhofhololi Tsanwani
 *  Student number: 261021222
 */
package atm;

import java.awt.BorderLayout;
import java.awt.Color;
import java.awt.Font;
import java.awt.GridLayout;
import java.awt.event.ActionEvent;
import java.awt.event.ActionListener;
import java.awt.event.WindowAdapter;
import java.awt.event.WindowEvent;
import javax.swing.BorderFactory;
import javax.swing.JButton;
import javax.swing.JFrame;
import javax.swing.JLabel;
import javax.swing.JOptionPane;
import javax.swing.JPanel;
import javax.swing.SwingConstants;

public class MainMenuFrame extends JFrame implements ActionListener {

    private final Account account;       // The customer currently logged in
    private JButton withdrawButton;
    private JButton depositButton;
    private JButton balanceButton;
    private JButton exitButton;

    public MainMenuFrame(Account account) {
        this.account = account;
        initComponents();
    }

    private void initComponents() {
```

```
        setTitle("CPUT Smart ATM - Main Menu");
        setSize(520, 560);
        setResizable(false);
        // Intercept window close (X button) to prompt for session confirmation
        setDefaultCloseOperation(JFrame.DO_NOTHING_ON_CLOSE);
        addWindowListener(new WindowAdapter() {
            @Override
            public void windowClosing(WindowEvent e) {
                confirmExit();
            }
        });
        setLayout(new BorderLayout());
        getContentPane().setBackground(ATMStyle.LIGHT);

        add(ATMStyle.createHeader("Main Menu"), BorderLayout.NORTH);

        // Welcome panel displaying holder name, masked account number, and prompt
        JPanel welcomePanel = new JPanel(new GridLayout(3, 1, 0, 4));
        welcomePanel.setBackground(ATMStyle.LIGHT);
        welcomePanel.setBorder(BorderFactory.createEmptyBorder(22, 20, 6, 20));
        welcomePanel.add(centredLabel("Welcome, " + account.getHolderName(),
                ATMStyle.HEADING_FONT, ATMStyle.NAVY));
        welcomePanel.add(centredLabel("Account: " +
ATMStyle.maskAccountNumber(account.getAccountNumber()),
                ATMStyle.SMALL_FONT, ATMStyle.GREY));
        welcomePanel.add(centredLabel("Please choose a transaction", ATMStyle.TEXT_FONT,
ATMStyle.NAVY));

        // Four primary menu option buttons arranged in a 2x2 grid
        withdrawButton = ATMStyle.createButton("Withdraw", ATMStyle.BLUE);
        depositButton  = ATMStyle.createButton("Deposit", ATMStyle.BLUE);
        balanceButton  = ATMStyle.createButton("Check Balance", ATMStyle.BLUE);
        exitButton     = ATMStyle.createButton("Exit", ATMStyle.RED);

        JButton[] allButtons = {withdrawButton, depositButton, balanceButton, exitButton};
        JPanel buttonPanel = new JPanel(new GridLayout(2, 2, 20, 20));
        buttonPanel.setBackground(ATMStyle.LIGHT);
        buttonPanel.setBorder(BorderFactory.createEmptyBorder(20, 40, 40, 40));
        for (JButton button : allButtons) {
            button.setFont(ATMStyle.HEADING_FONT);
            button.addActionListener(this);        // This frame listens for button clicks
            buttonPanel.add(button);
        }

        JPanel centre = new JPanel(new BorderLayout());
        centre.setBackground(ATMStyle.LIGHT);
        centre.add(welcomePanel, BorderLayout.NORTH);
        centre.add(buttonPanel, BorderLayout.CENTER);
        add(centre, BorderLayout.CENTER);

        ATMStyle.centreOnScreen(this);
    }

    // Helper method to create centered labels
```

```
    private JLabel centredLabel(String text, Font font, Color colour) {
        JLabel label = new JLabel(text, SwingConstants.CENTER);
        label.setFont(font);
        label.setForeground(colour);
        return label;
    }

    @Override
    public void actionPerformed(ActionEvent e) {
        Object source = e.getSource();

        if (source == withdrawButton) {
            WithdrawFrame screen = new WithdrawFrame(this, account);
            openScreen(screen);
        } else if (source == depositButton) {
            DepositFrame screen = new DepositFrame(this, account);
            openScreen(screen);
        } else if (source == balanceButton) {
            BalanceFrame screen = new BalanceFrame(this, account);
            openScreen(screen);
        } else if (source == exitButton) {
            confirmExit();
        }
    }

    // Opens a specific transaction screen and hides the main menu until returning
    private void openScreen(JFrame screen) {
        screen.setVisible(true);
        this.setVisible(false);
    }

    // Prompts the user for confirmation before ending the session and closing the application
    private void confirmExit() {
        int choice = JOptionPane.showConfirmDialog(this,
                "Do you want to end your session and exit?",
                "Exit", JOptionPane.YES_NO_OPTION);

        if (choice == JOptionPane.YES_OPTION) {
            ATMStyle.showInfo(this, "Thank you for using " + ATMStyle.BANK_NAME
                    + ".\nPlease remember to take your card.");
            System.exit(0);          // Terminate program
        }
    }
}
```

B.10 WithdrawFrame.java

```
/*
 *  Author: Muvhofhololi Tsanwani
 *  Student Number: 261021222
 */

package atm;
```

```
import java.awt.BorderLayout;
import java.awt.Dimension;
import java.awt.GridBagConstraints;
import java.awt.GridBagLayout;
import java.awt.GridLayout;
import java.awt.Insets;
import java.awt.event.ActionEvent;
import java.awt.event.ActionListener;
import java.awt.event.WindowAdapter;
import java.awt.event.WindowEvent;
import javax.swing.BorderFactory;
import javax.swing.JButton;
import javax.swing.JFrame;
import javax.swing.JLabel;
import javax.swing.JPanel;
import javax.swing.JTextField;
import javax.swing.SwingConstants;

public class WithdrawFrame extends JFrame implements ActionListener
{

   private final MainMenuFrame menu;                    // Screen to return to
   private final Account account;                       // The logged-in customer
   private final int[] quickAmounts = {100, 200, 500, 1000, 2000, 5000};  // Quick cash preset options
   private JButton[] quickButtons;                      // Buttons for quick cash amounts
   private JTextField amountField;                      // Text field for custom withdrawal amounts
   private JButton withdrawButton;
   private JButton clearButton;
   private JButton backButton;
   private JLabel balanceLabel;                         // Displays the current account balance

   public WithdrawFrame(MainMenuFrame menu, Account account) {
      this.menu = menu;
      this.account = account;
      initComponents();
   }

   private void initComponents() {
      setTitle("CPUT Smart ATM - Withdraw");
      setSize(520, 640);
      setResizable(false);
      setDefaultCloseOperation(JFrame.DO_NOTHING_ON_CLOSE);

      // Override window close button (X) to return to the main menu instead of closing abruptly
      addWindowListener(new WindowAdapter() {
         @Override
         public void windowClosing(WindowEvent e) {
            goBack();
         }
      });
      setLayout(new BorderLayout());
      getContentPane().setBackground(ATMStyle.LIGHT);
```

```
        add(ATMStyle.createHeader("Withdraw Cash"), BorderLayout.NORTH);

        JPanel form = new JPanel(new GridBagLayout());
        form.setBackground(ATMStyle.LIGHT);
        form.setBorder(BorderFactory.createEmptyBorder(20, 40, 10, 40));
        GridBagConstraints gbc = new GridBagConstraints();
        gbc.gridx = 0;
        gbc.weightx = 1.0;
        gbc.fill = GridBagConstraints.HORIZONTAL;
        gbc.insets = new Insets(8, 0, 8, 0);

        // Balance display at the top of the form
        balanceLabel = new JLabel("", SwingConstants.CENTER);
        balanceLabel.setFont(ATMStyle.HEADING_FONT);
        balanceLabel.setForeground(ATMStyle.GREEN);
        updateBalanceLabel();
        gbc.gridy = 0;
        form.add(balanceLabel, gbc);

        // Quick cash preset buttons arranged in a 2x3 grid
        gbc.gridy = 1;
        form.add(ATMStyle.createLabel("Quick cash - tap an amount:", ATMStyle.TEXT_FONT,
ATMStyle.NAVY), gbc);

        JPanel quickPanel = new JPanel(new GridLayout(2, 3, 10, 10));
        quickPanel.setBackground(ATMStyle.LIGHT);
        quickButtons = new JButton[quickAmounts.length];
        for (int i = 0; i < quickAmounts.length; i++) {
            quickButtons[i] = ATMStyle.createButton(ATMStyle.formatMoney(quickAmounts[i]),
ATMStyle.BLUE);
            quickButtons[i].addActionListener(this);
            quickPanel.add(quickButtons[i]);
        }
        gbc.gridy = 2;
        form.add(quickPanel, gbc);

        // Custom amount input prompt referencing note denomination rules
        gbc.gridy = 3;
        gbc.insets = new Insets(20, 0, 6, 0);
        form.add(ATMStyle.createLabel("Or type another amount (multiples of R10):",
              ATMStyle.TEXT_FONT, ATMStyle.NAVY), gbc);

        amountField = new JTextField();
        amountField.setFont(ATMStyle.HEADING_FONT);
        amountField.setPreferredSize(new Dimension(300, 44));
        gbc.gridy = 4;
        gbc.insets = new Insets(6, 0, 6, 0);
        form.add(amountField, gbc);

        // Action buttons
        withdrawButton = ATMStyle.createButton("Withdraw", ATMStyle.GREEN);
        clearButton    = ATMStyle.createButton("Clear", ATMStyle.GREY);
        backButton     = ATMStyle.createButton("Back", ATMStyle.RED);
        withdrawButton.addActionListener(this);
```

```
        clearButton.addActionListener(this);
        backButton.addActionListener(this);

        JPanel buttonPanel = new JPanel(new GridLayout(1, 3, 12, 0));
        buttonPanel.setBackground(ATMStyle.LIGHT);
        buttonPanel.add(withdrawButton);
        buttonPanel.add(clearButton);
        buttonPanel.add(backButton);
        gbc.gridy = 5;
        gbc.insets = new Insets(18, 0, 0, 0);
        form.add(buttonPanel, gbc);

        // Vertical glue constraint to push form components toward the top
        gbc.gridy = 6;
        gbc.weighty = 1.0;
        form.add(new JLabel(), gbc);

        add(form, BorderLayout.CENTER);
        getRootPane().setDefaultButton(withdrawButton);   // Pressing Enter triggers the withdraw
button
        ATMStyle.centreOnScreen(this);
    }

    @Override
    public void actionPerformed(ActionEvent e) {
        Object source = e.getSource();

        if (source == withdrawButton) {
            processWithdrawal();
        } else if (source == clearButton) {
            amountField.setText("");
        } else if (source == backButton) {
            goBack();
        } else {
            // Identify which quick cash button was clicked and populate the input field
            for (int i = 0; i < quickButtons.length; i++) {
                if (source == quickButtons[i]) {
                    amountField.setText(String.valueOf(quickAmounts[i]));
                }
            }
        }
    }

    // Validates input, enforces withdrawal limits and note rules, and updates account balance
    private void processWithdrawal() {
        String text = amountField.getText().trim();

        if (text.isEmpty()) {
            ATMStyle.showError(this, "Please enter an amount first.");
            return;
        }
        if (!ATMStyle.isAmountText(text)) {
            ATMStyle.showError(this, "Please use numbers only, for example 500 or 250.50.");
            return;
```

```
        }

        try {
            double amount = Double.parseDouble(text);
            account.withdraw(amount);              // Enforces transaction limits, note multiples, and funds

            updateBalanceLabel();
            amountField.setText("");
            ATMStyle.showInfo(this, "Withdrawal successful!\n"
                    + "Please take your cash: " + ATMStyle.formatMoney(amount) + "\n"
                    + "Remaining balance: " + ATMStyle.formatMoney(account.getBalance()));

        } catch (NumberFormatException ex) {
            ATMStyle.showError(this, "That is not a valid number.");
        } catch (InvalidAmountException ex) {
            ATMStyle.showError(this, ex.getMessage());
        } catch (InsufficientFundsException ex) {
            ATMStyle.showError(this, ex.getMessage());
        }
    }

    // Refreshes the balance label with the latest account balance
    private void updateBalanceLabel() {
        balanceLabel.setText("Available balance: " + ATMStyle.formatMoney(account.getBalance()));
    }

    // Returns to the main menu screen and disposes of the withdrawal frame
    private void goBack() {
        menu.setVisible(true);
        dispose();
    }
}
```

B.11 DepositFrame.java

```
/*
 *  Author;     Vhukhudo Ravele
 *  Student No.; 260048496
 */
package atm;

import java.awt.BorderLayout;
import java.awt.Dimension;
import java.awt.GridBagConstraints;
import java.awt.GridBagLayout;
import java.awt.GridLayout;
import java.awt.Insets;
import java.awt.event.ActionEvent;
import java.awt.event.ActionListener;
import java.awt.event.WindowAdapter;
import java.awt.event.WindowEvent;
import javax.swing.BorderFactory;
import javax.swing.JButton;
```

```
import javax.swing.JFrame;
import javax.swing.JLabel;
import javax.swing.JPanel;
import javax.swing.JTextField;
import javax.swing.SwingConstants;

public class DepositFrame extends JFrame implements ActionListener {

    private final MainMenuFrame menu;
    private final Account account;
    private JTextField amountField;
    private JButton depositButton;
    private JButton clearButton;
    private JButton backButton;
    private JLabel balanceLabel;

    public DepositFrame(MainMenuFrame menu, Account account) {
        this.menu = menu;
        this.account = account;
        initComponents();
    }

    private void initComponents() {
        setTitle("CPUT Smart ATM - Deposit");
        setSize(520, 520);
        setResizable(false);
        setDefaultCloseOperation(JFrame.DO_NOTHING_ON_CLOSE);

        // Override window close button (X) to return to the menu instead of exiting
        addWindowListener(new WindowAdapter() {
            @Override
            public void windowClosing(WindowEvent e) {
                goBack();
            }
        });
        setLayout(new BorderLayout());
        getContentPane().setBackground(ATMStyle.LIGHT);

        add(ATMStyle.createHeader("Deposit Cash"), BorderLayout.NORTH);

        JPanel form = new JPanel(new GridBagLayout());
        form.setBackground(ATMStyle.LIGHT);
        form.setBorder(BorderFactory.createEmptyBorder(30, 40, 10, 40));
        GridBagConstraints gbc = new GridBagConstraints();
        gbc.gridx = 0;
        gbc.weightx = 1.0;
        gbc.fill = GridBagConstraints.HORIZONTAL;
        gbc.insets = new Insets(8, 0, 8, 0);

        // Balance label at the top of the form
        balanceLabel = new JLabel("", SwingConstants.CENTER);
        balanceLabel.setFont(ATMStyle.HEADING_FONT);
        balanceLabel.setForeground(ATMStyle.GREEN);
        updateBalanceLabel();
```

```
        gbc.gridy = 0;
        form.add(balanceLabel, gbc);

        // Instruction label showing max deposit limits
        gbc.gridy = 1;
        gbc.insets = new Insets(24, 0, 6, 0);
        form.add(ATMStyle.createLabel("Enter the amount to deposit (max "
                + ATMStyle.formatMoney(Account.MAX_DEPOSIT) + "):",
                ATMStyle.TEXT_FONT, ATMStyle.NAVY), gbc);

        // Amount input text field
        amountField = new JTextField();
        amountField.setFont(ATMStyle.HEADING_FONT);
        amountField.setPreferredSize(new Dimension(300, 44));
        gbc.gridy = 2;
        gbc.insets = new Insets(6, 0, 6, 0);
        form.add(amountField, gbc);

        depositButton = ATMStyle.createButton("Deposit", ATMStyle.GREEN);
        clearButton   = ATMStyle.createButton("Clear", ATMStyle.GREY);
        backButton    = ATMStyle.createButton("Back", ATMStyle.RED);
        depositButton.addActionListener(this);
        clearButton.addActionListener(this);
        backButton.addActionListener(this);

        JPanel buttonPanel = new JPanel(new GridLayout(1, 3, 12, 0));
        buttonPanel.setBackground(ATMStyle.LIGHT);
        buttonPanel.add(depositButton);
        buttonPanel.add(clearButton);
        buttonPanel.add(backButton);
        gbc.gridy = 3;
        gbc.insets = new Insets(22, 0, 0, 0);
        form.add(buttonPanel, gbc);

        // Glue component to push form contents toward the top
        gbc.gridy = 4;
        gbc.weighty = 1.0;
        form.add(new JLabel(), gbc);

        add(form, BorderLayout.CENTER);
        getRootPane().setDefaultButton(depositButton);
        ATMStyle.centreOnScreen(this);
    }

    @Override
    public void actionPerformed(ActionEvent e) {
        Object source = e.getSource();

        if (source == depositButton) {
            processDeposit();
        } else if (source == clearButton) {
            amountField.setText("");
        } else if (source == backButton) {
            goBack();
```

```
        }
    }

    // Validates user input, processes the cash deposit, and handles potential exceptions
    private void processDeposit() {
        String text = amountField.getText().trim();

        if (text.isEmpty()) {
            ATMStyle.showError(this, "Please enter an amount first.");
            return;
        }
        if (!ATMStyle.isAmountText(text)) {
            ATMStyle.showError(this, "Please use numbers only, for example 500 or 250.50.");
            return;
        }

        try {
            double amount = Double.parseDouble(text);
            account.deposit(amount); // Throws InvalidAmountException if limits/rules are violated

            updateBalanceLabel();
            amountField.setText("");
            ATMStyle.showInfo(this, "Deposit successful!\n"
                    + "Amount deposited: " + ATMStyle.formatMoney(amount) + "\n"
                    + "New balance: " + ATMStyle.formatMoney(account.getBalance()));

        } catch (NumberFormatException ex) {
            ATMStyle.showError(this, "That is not a valid number.");
        } catch (InvalidAmountException ex) {
            ATMStyle.showError(this, ex.getMessage());
        }
    }

    // Refreshes the balance label with the latest account balance
    private void updateBalanceLabel() {
        balanceLabel.setText("Available balance: " + ATMStyle.formatMoney(account.getBalance()));
    }

    // Returns to the main menu screen and disposes of the deposit frame
    private void goBack() {
        menu.setVisible(true);
        dispose();
    }
}
```

B.12 BalanceFrame.java

```
/*
 *  AUTHOR: Athenkosi Khau
 *  STUDENT NUMBER: 215107020
 */
package atm;
```

```
import java.awt.BorderLayout;
import java.awt.Color;
import java.awt.Font;
import java.awt.GridLayout;
import java.awt.Insets;
import java.awt.event.ActionEvent;
import java.awt.event.ActionListener;
import java.awt.event.WindowAdapter;
import java.awt.event.WindowEvent;
import java.util.ArrayList;
import javax.swing.BorderFactory;
import javax.swing.JButton;
import javax.swing.JFrame;
import javax.swing.JLabel;
import javax.swing.JPanel;
import javax.swing.JScrollPane;
import javax.swing.JTextArea;
import javax.swing.SwingConstants;

public class BalanceFrame extends JFrame implements ActionListener {

   private final MainMenuFrame menu;
   private final Account account;
   private JTextArea statementArea;
   private JButton backButton;

   public BalanceFrame(MainMenuFrame menu, Account account) {
      this.menu = menu;
      this.account = account;
      initComponents();
      loadStatement(); // Populates the mini-statement text area on load
   }

   private void initComponents() {
      setTitle("CPUT Smart ATM - Check Balance");
      setSize(520, 640);
      setResizable(false);
      setDefaultCloseOperation(JFrame.DO_NOTHING_ON_CLOSE);

      // Override window close button (X) to return to the main menu instead of closing the
application
      addWindowListener(new WindowAdapter() {
         @Override
         public void windowClosing(WindowEvent e) {
            goBack();
         }
      });
      setLayout(new BorderLayout());
      getContentPane().setBackground(ATMStyle.LIGHT);

      add(ATMStyle.createHeader("Account Balance"), BorderLayout.NORTH);

      // Panel displaying account holder details, masked account number, and current balance
      JPanel infoPanel = new JPanel(new GridLayout(4, 1, 0, 4));
```

```
        infoPanel.setBackground(ATMStyle.LIGHT);
        infoPanel.setBorder(BorderFactory.createEmptyBorder(22, 20, 10, 20));

        infoPanel.add(centredLabel(account.getHolderName(), ATMStyle.HEADING_FONT,
ATMStyle.NAVY));
        infoPanel.add(centredLabel("Account: " +
ATMStyle.maskAccountNumber(account.getAccountNumber()),
                ATMStyle.SMALL_FONT, ATMStyle.GREY));
        infoPanel.add(centredLabel("Available balance", ATMStyle.TEXT_FONT, ATMStyle.NAVY));
        infoPanel.add(centredLabel(ATMStyle.formatMoney(account.getBalance()),
                ATMStyle.MONEY_FONT, ATMStyle.GREEN));

        // Setup scrollable text area for the mini statement
        statementArea = new JTextArea();
        statementArea.setEditable(false);
        statementArea.setFont(new Font(Font.MONOSPACED, Font.PLAIN, 11));
        statementArea.setMargin(new Insets(8, 8, 8, 8));

        JScrollPane scrollPane = new JScrollPane(statementArea);
        scrollPane.setBorder(BorderFactory.createTitledBorder("Mini statement (last 5 transactions)"));

        JPanel statementPanel = new JPanel(new BorderLayout());
        statementPanel.setBackground(ATMStyle.LIGHT);
        statementPanel.setBorder(BorderFactory.createEmptyBorder(0, 25, 10, 25));
        statementPanel.add(scrollPane, BorderLayout.CENTER);

        backButton = ATMStyle.createButton("Back", ATMStyle.RED);
        backButton.addActionListener(this);
        JPanel bottomPanel = new JPanel();
        bottomPanel.setBackground(ATMStyle.LIGHT);
        bottomPanel.setBorder(BorderFactory.createEmptyBorder(5, 0, 20, 0));
        bottomPanel.add(backButton);

        JPanel centre = new JPanel(new BorderLayout());
        centre.setBackground(ATMStyle.LIGHT);
        centre.add(infoPanel, BorderLayout.NORTH);
        centre.add(statementPanel, BorderLayout.CENTER);
        centre.add(bottomPanel, BorderLayout.SOUTH);
        add(centre, BorderLayout.CENTER);

        getRootPane().setDefaultButton(backButton);
        ATMStyle.centreOnScreen(this);
    }

    // Helper method to create centered labels with specified font and color
    private JLabel centredLabel(String text, Font font, Color colour) {
        JLabel label = new JLabel(text, SwingConstants.CENTER);
        label.setFont(font);
        label.setForeground(colour);
        return label;
    }

    // Fetches recent transactions from the account and displays them in the mini statement box
    private void loadStatement() {
```

```
        ArrayList<Transaction> recent = account.getLastTransactions(5);

        if (recent.isEmpty()) {
            statementArea.setText("No transactions yet.");
            return;
        }

        String text = "";
        for (Transaction transaction : recent) {
            text = text + transaction.toString() + "\n";
        }
        statementArea.setText(text);
    }

    @Override
    public void actionPerformed(ActionEvent e) {
        if (e.getSource() == backButton) {
            goBack();
        }
    }

    // Returns to the main menu screen and disposes of the balance frame
    private void goBack() {
        menu.setVisible(true);
        dispose();
    }
}
```

B.13 ATMLogicTest.java

```
/*
 *  Author      : Siyolise Mhlauli
 *  Student No. : 261295438
 */
package atm;

public class ATMLogicTest {

    private static int passed = 0;
    private static int failed = 0;

    // Helper method to log test results and update pass/fail counters
    private static void check(String description, boolean condition) {
        if (condition) {
            passed++;
            System.out.println("PASS  " + description);
        } else {
            failed++;
            System.out.println("FAIL  " + description);
        }
    }

    // Floating-point comparison helper with a small tolerance for cent rounding
```

```
private static boolean sameMoney(double a, double b) {
   return Math.abs(a - b) < 0.005;
}

public static void main(String[] args) {

   System.out.println("=== Account tests ===");
   Account account = new Account("1111111111", "Test Customer", "1234");
   check("T01 new account starts with R10 000.00", sameMoney(account.getBalance(), 10000.00));

   try {
      account.deposit(500);
      check("T02 deposit R500 gives R10 500.00", sameMoney(account.getBalance(), 10500.00));
      account.deposit(250.50);
      check("T03 deposit R250.50 gives R10 750.50", sameMoney(account.getBalance(), 10750.50));
      account.withdraw(300);
      check("T04 withdraw R300 gives R10 450.50", sameMoney(account.getBalance(), 10450.50));
   } catch (Exception e) {
      check("T02-T04 valid deposits and withdrawals must not throw: " + e.getMessage(), false);
   }

   double before = account.getBalance();

   // Testing invalid withdrawal amounts and business constraints
   try {
      account.withdraw(15);
      check("T05 withdraw R15 is rejected (not a multiple of R10)", false);
   } catch (InvalidAmountException e) {
      check("T05 withdraw R15 is rejected (not a multiple of R10)", true);
   } catch (InsufficientFundsException e) {
      check("T05 withdraw R15 is rejected (not a multiple of R10)", false);
   }

   try {
      account.withdraw(6000);
      check("T06 withdraw R6 000 is rejected (above R5 000 limit)", false);
   } catch (InvalidAmountException e) {
      check("T06 withdraw R6 000 is rejected (above R5 000 limit)", true);
   } catch (InsufficientFundsException e) {
      check("T06 withdraw R6 000 is rejected (above R5 000 limit)", false);
   }

   try {
      account.withdraw(0);
      check("T07 withdraw R0 is rejected", false);
   } catch (InvalidAmountException e) {
      check("T07 withdraw R0 is rejected", true);
   } catch (InsufficientFundsException e) {
      check("T07 withdraw R0 is rejected", false);
   }

   try {
      account.withdraw(-50);
      check("T08 withdraw a negative amount is rejected", false);
```

```
        } catch (InvalidAmountException e) {
            check("T08 withdraw a negative amount is rejected", true);
        } catch (InsufficientFundsException e) {
            check("T08 withdraw a negative amount is rejected", false);
        }

        check("T09 rejected withdrawals did not change the balance",
sameMoney(account.getBalance(), before));

        // Testing insufficient funds handling and shortfall calculation
        Account poor = new Account("2222222222", "Low Balance", "1111", 100.00);
        try {
            poor.withdraw(500);
            check("T10 withdraw R500 from R100 throws InsufficientFundsException", false);
        } catch (InsufficientFundsException e) {
            check("T10 withdraw R500 from R100 throws InsufficientFundsException", true);
            check("T11 shortfall is R400.00", sameMoney(e.getShortfall(), 400.00));
        } catch (InvalidAmountException e) {
            check("T10 withdraw R500 from R100 throws InsufficientFundsException", false);
        }
        check("T12 balance stays R100.00 after the failed withdrawal", sameMoney(poor.getBalance(),
100.00));

        // Testing edge cases for deposits using an array loop
        double[] badDeposits = {0, -20, 25000, 10.123};
        String[] badNames = {"R0", "-R20", "R25 000 (above R20 000 limit)", "R10.123 (3 decimals)"};
        for (int i = 0; i < badDeposits.length; i++) {
            try {
                account.deposit(badDeposits[i]);
                check("T" + (13 + i) + " deposit " + badNames[i] + " is rejected", false);
            } catch (InvalidAmountException e) {
                check("T" + (13 + i) + " deposit " + badNames[i] + " is rejected", true);
            }
        }

        try {
            Account rich = new Account("3333333333", "Rich Customer", "9999", 20000.00);
            rich.withdraw(5000);
            check("T17 withdraw exactly R5 000 is allowed", sameMoney(rich.getBalance(), 15000.00));
        } catch (Exception e) {
            check("T17 withdraw exactly R5 000 is allowed", false);
        }

        check("T18 history holds the 3 successful transactions", account.getLastTransactions(5).size() ==
3);

        check("T19 correct PIN is accepted", account.checkPin("1234"));
        check("T20 wrong PIN is rejected", !account.checkPin("0000"));

        System.out.println();
        System.out.println("=== Bank tests ===");
        Bank bank = new Bank();
        check("T21 bank loads 3 demo accounts", bank.getAccountCount() == 3);
        check("T22 unknown account number is not found",
```

```
            bank.authenticate("0000000000", "1234") == Bank.AUTH_NO_ACCOUNT);
        check("T23 correct login succeeds",
            bank.authenticate("1234567890", "1234") == Bank.AUTH_SUCCESS);

        // Verifying that entering 3 incorrect PINs locks the account
        int first = bank.authenticate("0987654321", "0000");
        int second = bank.authenticate("0987654321", "0000");
        int third = bank.authenticate("0987654321", "0000");
        int afterBlock = bank.authenticate("0987654321", "4321");
        check("T24 3 wrong PINs block the card (even the correct PIN is then refused)",
            first == Bank.AUTH_WRONG_PIN && second == Bank.AUTH_WRONG_PIN
            && third == Bank.AUTH_LOCKED && afterBlock == Bank.AUTH_LOCKED);

        System.out.println();
        System.out.println("Result: " + passed + " passed, " + failed + " failed.");
        if (failed > 0) {
           System.exit(1);
        }
    }
}
```